

TEMA 7 — ASPECTOS FUNCIONALES

1) Punteros a función en C

Un puntero a función es una variable que guarda la dirección de una función.

Permite:

- Pasar funciones como parámetros (callbacks).
- Elegir qué función ejecutar en tiempo de ejecución.
- Guardarlas en estructuras (tablas de funciones).

Condición importante:

El puntero debe tener la misma firma (parámetros y retorno) que la función.

Conceptualmente, es “tratar código como dato”, pero a nivel bajo (direcciones).

2) Qué es una lambda

Una función lambda es una función anónima que se puede:

- asignar a una variable,
- pasar como argumento,
- devolver como resultado.

Se usa para expresar comportamiento de forma compacta.

Las lambdas existen, pero siempre tienen un tipo asociado: una interfaz funcional.

3) Paradigma funcional y lenguaje multiparadigma

El paradigma funcional pone el foco en:

- construir programas como composición de funciones,
- evitar (en lo posible) estado mutable y efectos secundarios,
- preferir funciones puras: mismos argumentos $\Rightarrow$ mismo resultado.

Java se considera multiparadigma porque combina:

- orientación a objetos,
- estilo imperativo,
- y estilo funcional (lambdas, streams, etc.).

4) Sintaxis básica de lambdas en Java

Forma general:

(params) -> expresion

(params) -> { bloque }

Detalles:

- Si es una sola expresión, no necesita llaves ni return.
- Si hay varias instrucciones, usas {} y return si devuelves algo.

5) Funciones como parámetros (orden superior)

Una idea esencial funcional es que un método reciba una función para decidir “qué hacer”.

Ejemplo conceptual: transformar(texto, funcion)

El método no conoce la transformación exacta.

Solo aplica la función recibida.

6) Lambdas inline (definir una función en la llamada)

Además de guardar la lambda en una variable, puedes definirla justo cuando la pasas como argumento.

Ventaja: más conciso cuando se usa una sola vez.

7) Closures (cierres)

Un closure es una función que “captura” variables del entorno donde se creó.

En Java hay una restricción:

las variables locales capturadas deben ser efectivamente finales (no modificadas después de inicializar).

Esto evita problemas con estado mutable.

8) Lambda vs puntero a función (diferencia conceptual)

En C, un puntero a función es básicamente una dirección.

No “arrastra” contexto.

Una lambda (en muchos lenguajes) puede ser un closure:

encapsula código + acceso a valores del entorno.

Además:

En Java, el compilador controla tipos mediante interfaces funcionales.

En C, el control de firmas es más manual y de bajo nivel.

9) Devolver funciones (funciones que generan funciones)

Otra idea potente funcional:

un método puede devolver una función.

10) Interfaz funcional (el tipo de una lambda en Java)

Una interfaz funcional es una interfaz con:

- un único método abstracto.

Puede tener además:

- métodos default,
- static,
- métodos heredados de Object,

sin dejar de ser funcional.

@FunctionalInterface es recomendable para que el compilador garantice la condición.

11) Definir una interfaz funcional propia (y generalizarla con genéricos)

Puedes crear una interfaz tipo Transformador para representar “entrada → salida”.

Luego generalizarla como Transformador<T, R> para cualquier tipo.

Conceptualmente equivale a una interfaz estándar ya existente: Function<T, R>.

12) Interfaces funcionales estándar típicas en Java

En java.util.function aparecen patrones comunes:

Function<T,R>: transforma $T \rightarrow R$

Predicate<T>: $T \rightarrow \text{boolean}$ (condición)

Consumer<T>: consume T, sin retorno

Supplier<T>: no recibe, produce T

13) forEach como versión funcional del bucle

forEach recorre una colección aplicando una función a cada elemento.

La función que recibe es un Consumer (consume el elemento).

Permite escribir recorrido de forma más declarativa: “haz esto con cada elemento”.

14) PECS aplicado a consumidores (por qué Consumer<? super T>)

Cuando una función consume valores T, es útil permitir que acepte T o un supertipo:

si tengo Integer, un consumidor de Number o de Object también sirve.

Por eso aparece ? super T en consumidores: es más flexible sin perder seguridad.

15) Referencias a método (method references)

Son una forma compacta de usar un método existente como función.

En Java se escriben con `::` y encajan con una interfaz funcional compatible.

En JavaScript, al extraer un método como referencia hay que cuidar el `this` (a veces se usa `bind` para conservar el contexto).

Tipos habituales en Java:

- método estático (`Clase::metodo`)
- constructor (`Clase::new`)
- método de una instancia concreta (`objeto::metodo`)
- método de instancia “sobre cualquier instancia” (`Clase::metodo` aplicado vía parámetro)
-

16) Ordenación funcional con comparadores

Se puede ordenar una lista pasando una función comparadora:

- versión “manual” (lambda con lógica `if`)
- versión más declarativa encadenando criterios con `Comparator.comparing(...).thenComparing(...)`